

Special Participation C1: RNN Notebook Refactoring Report

Sarvagya Somvanshi, Arnav Dalal, Kabir Shah

1 Overview

We refactored the `q_rnn_and_grad.ipynb` teaching notebook into a modular Python package. The objective was to align the assignment with professional software engineering practices while preserving its pedagogical structure. By decoupling logic, tests, and utilities, we improved maintainability and readability, strictly adhering to PEP 8 (style), PEP 257 (docstrings), and PEP 484 (type hinting).

2 Architecture

We transitioned from a monolithic notebook to a component-based structure:

```
special-c1/
|-- q_rnn_and_grad.ipynb    # Main student interface
|-- utils.py                # Helper functions (data gen, training loops)
|-- tests.py                # Unit tests and assertions
|-- img/                    # Architecture diagrams & assets
|-- requirements.txt        # (Implicit) Dependencies list
|-- readme.md               # Project documentation
```

3 Refactoring Methodology

3.1 Modularization Strategy

The original notebook interleaved class definitions, global utilities, and inline assertions. We separated these concerns to simulate a production environment:

- **Extraction of Utilities:** Functions such as `generate_batch`, `biggest_eig_magnitude`, and the `train` loop were moved to `utils.py` to declutter the notebook global namespace.
- **Test Encapsulation:** Inline assertions (e.g., `assert ...`) were wrapped into callable functions within `tests.py`. This allows students to verify components (e.g., `test_rnn_layer(RNNLayer)`) without seeing the assertion logic immediately, preventing "programming to the test."
- **Notebook Focus:** The notebook now strictly contains narrative, core model definitions (`RNNLayer`, `RecurrentRegressionModel`), and the `GradientVisualizer` (kept inline due to its dependency on interactive widget plotting).

3.2 Code Standardization (PEPs)

- **Type Hinting (PEP 484):** Added signatures to all methods.
 - *Before:* `def forward(self, x):`
 - *After:* `def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:`
- **Documentation (PEP 257):** Replaced sparse inline comments with NumPy-style docstrings, including Parameters, Returns, and mathematical context (e.g., $h_t = \tanh(W_{hh}h_{t-1} + W_{ih}x_t + b)$).
- **Style Naming (PEP 8):**
 - Converted 2-space indentation to 4-space.
 - Renamed variables to match PyTorch conventions: `th` $\rightarrow$ `torch`, `h_list` $\rightarrow$ `hidden_states`, and weight matrices (W_{ih} , W_{hh}) for clarity.

4 Technical Implementation Details

We introduced specific mechanisms to ensure the refactored code runs smoothly in a notebook environment:

- **Autoreload Magic:** Added `%load_ext autoreload` and `%autoreload 2` to the notebook setup. This ensures that changes made to `utils.py` or `tests.py` are immediately reflected in the notebook without restarting the kernel.
- **Environment Agnosticism:** Implemented a try-except block to detect Google Colab environments. This conditionally enables the custom widget manager required for the gradient visualization tools.
- **Reproducibility:** Centralized the random seed initialization. The `utils.py` module now enforces `torch.manual_seed(42)` and `np.random.seed(42)` to ensure deterministic outputs for grading.

4.1 Code Samples: Before vs. After

Original RNNLayer (Before):

```
class RNNLayer(nn.Module):
    def __init__(self, input_size, hidden_size, nonlinearity=th.tanh):
        super().__init__()
        # 2-space indent, no type hints, no docstring

    def forward(self, x):
        h_list = [] # Abbreviated variable name
```

Refactored RNNLayer (After):

```
class RNNLayer(nn.Module):
    """Single-layer Elman RNN implementation.

    Implements:  $h_t = \sigma(W_{hh} @ h_{t-1} + W_{ih} @ x_t + b)$ 
    """
```

```

def __init__(
    self,
    input_size: int,
    hidden_size: int,
    nonlinearity: Callable[[torch.Tensor], torch.Tensor] = torch.tanh
) -> None:
    """Initialize a single RNN layer.

    Args:
        input_size: Data input feature dimension.
        hidden_size: RNN hidden state size.
        nonlinearity: Activation function. Default: tanh.
    """
    super().__init__()

def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
    hidden_states = [] # Descriptive variable name

```

5 Comprehensive Refactoring Summary

Aspect	Original	Refactored	Reference
File Structure	Single notebook	notebook + utils.py + tests.py	Separation of concerns
Indentation	2-space	4-space	PEP 8
Import Alias	import torch as th	import torch	Convention
Variable Names	h_list	hidden_states	PEP 8
Type Hints	None	Full annotations	PEP 484
Docstrings	Sparse comments	NumPy-style	PEP 257
Tests	Inline assert	Wrapped in tests.py	Modularity
Constants	Magic numbers	RANDOM_SEED = 42	Google Style
Environment	Colab-only	Try/except detection	Cross-platform

6 Conclusion

The refactoring of the RNN assignment successfully bridges the gap between pedagogical simplicity and engineering rigor. By modularizing the codebase, we have created a more robust, deterministic, and maintainable assignment platform. The new structure not only simplifies grading via the isolated ‘tests.py’ module but also provides students with a clearer, less distracted environment to master the complexities of sequence modeling.

7 LLM Usage Statement

We utilized Claude to assist with the refactoring process. The LLM was used for:

1. **Docstring Expansion:** Generating draft NumPy-style docstrings based on the mathematical logic in the original code.
2. **Test Logic Extraction:** identifying inline assertions and wrapping them into the tests.py module functions.

3. **Style Enforcement:** Automating the indentation correction (2-space to 4-space) and type hint insertion.